Paper 1: A Survey on the Evolution of Stream Processing Systems (2024)

1. Introduction and Background

The paper reviews the development of stream processing systems from 2000 to 2023. These systems process continuous data in real time and are widely used in monitoring, fraud detection, traffic control and cloud applications.

2. Problem Statement

Stream processing systems have evolved significantly but their designs differ in data management, state handling, fault tolerance and scalability. Understanding these changes and remaining challenges is important for future development.

3. Objectives

- Review the evolution of stream processing systems.
- Compare the first and second generations of systems.
- Identify major developments and open research problems.

4. Proposed Approach or Methodology

The authors conduct a comprehensive literature survey covering out of order data management, state management, fault tolerance, high availability, load management, elasticity and system reconfiguration.

5. Key Findings and Contributions

- Explains the major developments in stream processing.
- Organizes research according to important system functions.
- Highlights how modern systems support more advanced streaming applications.

6. Limitations and Future Research

The authors identify open challenges in managing out-of-order data, maintaining state, handling failures and adapting systems to changing workloads. The complexity of processing late-arriving data at scale also remains a concern.

7. Conclusion

The paper provides a comprehensive understanding of how stream processing systems have evolved and identifies areas requiring further research.

Paper 2: A Survey on Transactional Stream Processing (2023)

1. Introduction and Background

The paper examines transactional stream processing (TSP) which combines real-time data processing with database transaction guarantees. These guarantees include atomicity, consistency, isolation and durability (ACID).

2. Problem Statement

Traditional stream processing systems often lack strong transactional guarantees and consistent access to shared data. Integrating transactional functionality with real-time processing introduces complex design challenges.

3. Objectives

- Explain the concept and requirements of TSP.
- Review existing TSP solutions and their differences.
- Identify design challenges and future research directions.

4. Proposed Approach or Methodology

The authors conduct a survey of existing transactional stream processing systems. They examine their requirements, implementation approaches, application characteristics and design trade-offs.

5. Key Findings and Contributions

- Explains how transactional guarantees can be combined with stream processing.
- Compares different approaches to transactional stream processing.
- Highlights the trade-offs between application requirements, performance and transaction correctness.

6. Limitations and Future Research

The authors emphasize that no universally accepted standard approach exists. Existing solutions are designed for particular application characteristics creating a need for more flexible and broadly applicable systems.

7. Conclusion

The paper provides a foundation for understanding transactional stream processing and identifies opportunities to develop more efficient and adaptable TSP systems.

Paper 3: A Contemporary and Comprehensive Survey on Streaming Tensor Decomposition (2023)

1. Introduction and Background

The paper reviews streaming tensor decomposition also known as tensor tracking. Tensor decomposition represents multidimensional data using simpler components. Streaming methods update these components as new data arrives.

2. Problem Statement

Applying tensor decomposition to continuous data streams is challenging because the data size can grow indefinitely, patterns may change over time and incoming data may be uncertain or imperfect.

3. Objectives

- Review existing streaming tensor decomposition methods.
- Classify methods according to tensor formats and technical approaches.
- Identify current challenges and future research directions.

4. Proposed Approach or Methodology

The authors conduct a comprehensive literature survey and classify existing methods into three main categories:

- Streaming CP decomposition.
- Streaming Tucker decomposition.
- Other tensor formats including tensor-train, t-SVD and block-term decomposition (BTD).

They further categorize the algorithms according to their optimization frameworks and model architectures.

5. Key Findings and Contributions

- Provides a structured classification of streaming tensor decomposition methods.
- Summarizes existing research trends and advances.
- Explains different approaches for updating tensor representations as data arrives.

6. Limitations and Future Research

The paper identifies challenges involving unbounded data streams, changing data patterns, concept drift, dataset shift and uncertainty. Developing algorithms that remain accurate and efficient under these conditions is an important research direction.

7. Conclusion

The survey organizes current tensor tracking research and provides guidance for developing more efficient and adaptive methods for multidimensional streaming data.

Paper 4: Self-Adaptive Analytical Querying over Schemaless Data Streams (2025)

1. Introduction and Background

The paper investigates analytical querying over schemaless data streams. In such streams, records may contain different attributes and structures. This type of data is common in big data platforms where timely analytical insights are important.

2. Problem Statement

Traditional streaming analytical techniques can be ineffective when records have heterogeneous structures. This makes it difficult to formulate useful multidimensional queries over continuously changing data.

3. Objectives

- Address the challenges of analyzing schemaless data streams.
- Automatically identify interesting analytical queries.
- Adapt query selection to the heterogeneous data in the current window.

4. Proposed Approach or Methodology

The authors propose a self-adaptive approach that formulates multidimensional analytical queries under a sliding-window model. The method automatically adapts to changes in data structure and selects interesting queries for the current window. Its efficiency and effectiveness are evaluated experimentally.

5. Key Findings and Contributions

- Introduces an approach specifically designed for heterogeneous, schemaless streams.
- Automatically adapts analytical query formulation to incoming data.
- Evaluates the proposed method in terms of efficiency and effectiveness.

6. Limitations and Future Research

The paper addresses a challenging data environment in which record structures are heterogeneous. Further investigation may be needed to understand how the approach performs under different stream characteristics and operational constraints. These are potential research directions not necessarily limitations explicitly stated by the authors.

7. Conclusion

The paper presents a self-adaptive approach to analytical querying that helps generate relevant queries over schemaless data streams without relying on a fixed data structure.

Paper 5: Scalable Transactional Stream Processing on Multicore Processors (2025)

1. Introduction and Background

The paper introduces MorphStream, a transactional stream processing engine designed for multicore processors. Transactional stream processing systems manage concurrent access to shared mutable states while processing continuous data.

2. Problem Statement

Existing transactional stream processing engines use scheduling approaches that may not adapt effectively to different workloads. Transaction conflicts, complex data dependencies and non-deterministic state access can limit performance.

3. Objectives

- Improve parallelism and performance on multicore processors.
- Adapt scheduling to different workload characteristics.
- Support non-deterministic state access and window-based operations.

4. Proposed Approach or Methodology

The authors propose MorphStream which uses a three-stage execution framework:

1. Planning: Identifies transaction dependencies.
2. Scheduling: Selects execution strategies based on workload characteristics.
3. Execution: Processes transactions while maintaining transactional guarantees.

The system also uses a stateful task precedence graph for non-deterministic state access and multi-versioned state management for window-based operations.

5. Key Findings and Contributions

- Introduces an adaptive execution framework for transactional stream processing.
- Supports non-deterministic state access and overlapping windows.
- Reports up to 3.4× higher throughput and 69.1% lower latency than state-of-the-art transactional stream processing engines in the evaluated scenarios.

6. Limitations and Future Research

The evaluation focuses on multicore processing. Extending the system to distributed environments presents further challenges, including maintaining consistency and managing communication overhead. These are potential directions for further research rather than confirmed limitations of the evaluation.

7. Conclusion

MorphStream demonstrates how adaptive scheduling and dependency-aware execution can improve transactional stream processing performance while maintaining transaction semantics.